Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Исследование рядов Тейлора»

Выполнил:
Студент 3825Б1ПМии1
Катютин З. В.

Проверила:

Бусько П.В.

Нижний Новгород
2026

Содержание

Введение 3
Постановка задачи 4
Руководство пользователя ..... **Ошибка! Закладка не определена.**
Описание программной реализации 5
Результаты экспериментов 12
Заключение 17
Литература 18
Приложение 19

Введение

В рамках данной лабораторной работы мною разработана программа, предназначенная для исследования сходимости рядов Тейлора для некоторых элементарных функций, с помощью складывания членов различными способами, а также с помощью погрешности, сравнивая полученное значение с точным значением из библиотеки.

Постановка задачи

Исследовать способы сложения членов рядов Тейлора. Проанализировать полученные данные и точность полученных измерений и сделать вывод о двух способах суммирования.

Руководство пользователя

При запуске программы активируется пользовательский интерфейс, содержащий перечень из четырех функций. Для продолжения пользователю необходимо выбрать одну из функций и ввести ее номер в консоль:

```
 ВЫЧИСЛЕНИЕ ФУНКЦИЙ ЧЕРЕЗ РЯД ТЕЙЛОРА
Выберите функцию для вычисления:
1 - e^x
2 - sin(x)
3 - cos(x)
4 - ln(1+x)
Ваш выбор:
```

После выполнения предыдущих действий пользователь должен ввести значение x, максимальное количество членов ряда и выбрать один из двух способов суммирования:

```
Введите x: 5
Введите максимальное количество членов ряда: 10

Выберите способ суммирования:
1 - Суммирование с начала (прямой порядок)
2 - Суммирование с конца (обратный порядок)
Ваш выбор: 1
```

После выполнения предыдущих действий в консоль выводятся результаты эксперимента: точное значение функции, взятое из библиотеки; полученное значение функции; погрешность; и таблицу из 3 столбцов, где в первом столбце находится количество слагаемых, во втором столбце значение (сумма членов ряда), в третьем столбце находится разница с эталонным значением функции.

```
Используется суммирование с начала
 РЕЗУЛЬТАТЫ
Точное значение функции: 148.4131591026

Последнее значение: 143.6894565697
Погрешность функции: 4.7237025329

Таблица:
Кол-во слагаемых | Значение         | Ошибка
-----------------|------------------|------------------
               1 |     1.0000000000 |   147.4131591026
               2 |     6.0000000000 |   142.4131591026
               3 |    18.5000000000 |   129.9131591026
               4 |    39.3333333333 |   109.0798257692
               5 |    65.3750000000 |    83.0381591026
               6 |    91.4166666667 |    56.9964924359
               7 |   113.1180555556 |    35.2951035470
               8 |   128.6190476190 |    19.7941114835
               9 |   138.3071676587 |    10.1059914438
              10 |   143.6894565697 |     4.7237025329
```

Описание программной реализации

Моя программа состоит из файла заголовков, содержащих прототипы функции и исходных файлов.

Файлы:

`Header.h` – содержит структуру и прототипы всех основных функций

```
struct Series {
    std::vector<double> values;      // Массив значений членов ряда
    std::vector<double> sums;        // Массив сумм
    std::vector<double> errors;      // Массив ошибок
    double exactValue;              // Точное значение функции
};

Series calculateEx(double x, int maxZnach);
Series calculateSin(double x, int maxZnach);
Series calculateCos(double x, int maxZnach);
Series calculateLn(double x, int maxZnach);
Series calculateSumForward(const Series& inputSeries, int maxZnach);
Series calculateSumBackward(const Series& inputSeries, int maxZnach);
void printResults(const Series& series, int maxTerms);
```

Рисунок 1 Структура Series и прототипы функций

`Functions.cpp` – содержит реализацию всех основных функций

`TaylorProject.cpp` – содержит функцию `main()`

`struct Series` – структура хранящая массив значений члена ряда, массив накопленных сумм, массив ошибок и точное значение функции

Функции:

`Series calculateEx(double x, int maxZnach)` – Вычисление членов ряда экспоненты

Принцип работы:

Функция использует рекуррентное соотношение для эффективного вычисления членов ряда без повторного вычисления факториалов.

Алгоритм:

1. Создает структуру res и вычисляет точное значение exp(x) с помощью стандартной функции
2. Выделяет память под три массива размером maxZnach
3. Вычисляет первый член ряда: $X_0 = 1$ (при n=0, $x^0/0! = 1$)
4. Записывает первый член в res.values[0], начальную сумму и погрешность
5. Для каждого следующего члена от i=1 до maxZnach применяет рекурентную формулу
6. Возвращает структуру с заполненными значениями членов ряда

`Series calculateSin(double x, int maxZnach)` - Вычисление членов ряда синуса

Принцип работы:

Функция использует знакочередующийся ряд с рекуррентным переходом между нечетными степенями.

Пошаговый алгоритм:

1. Вычисляет точное значение sin(x)
2. Выделяет память под массивы
3. Вычисляет первый член: $X_0 = x$ (при n=0: $x^1/1! = x$)
4. Для i от 1 до maxZnach - 1 использует рекуррентную формулу, где i = n+1, поэтому:
   2i соответствует 2n+2
   2i+1 соответствует 2n+3

```
Series calculateSin(double x, int maxZnach) {
    Series res;

    res.exactValue = sin(x);
    res.values.resize(maxZnach);
    res.sums.resize(maxZnach);
    res.errors.resize(maxZnach);

    // Первый член ряда (для n=0: x^1/1! = x)
    res.values[0] = x;
    res.sums[0] = res.values[0];
    res.errors[0] = abs(res.sums[0] - res.exactValue);

    // X {n+1} = -X_n * (x^2 / ((2n+2)(2n+3)))
    for (int i = 1; i < maxZnach; i++) {
        double factor = (x * x) / ((2 * i) * (2 * i + 1));
        res.values[i] = -res.values[i - 1] * factor;
    }
    return res;
}
```

Рисунок 2 Реализация функции подсчета значений для Синуса

`Series calculateCos(double x, int maxZnach)` – Вычисление членов ряда косинуса

Принцип работы:

Аналогичен ряду синуса, но использует четные степени.

Пошаговый алгоритм:

1. Вычисляет точное значение cos(x)
2. Выделяет память под массивы
3. Вычисляет первый член: $X_0 = 1$ (при n=0: $x^0/0! = 1$)
4. Использует рекуррентную формулу

`Series calculateLn(double x, int maxZnach)` – Вычисление членов ряда логарифма

Принцип работы:

Функция содержит проверку области определения и использует знакочередующийся ряд с линейным ростом знаменателя.

Пошаговый алгоритм

1. Проверяет условие x > -1, иначе выводит ошибку

2. Вычисляет точное значение log(1+x)
3. Выделяет память под массивы
4. Вычисляет первый член: $X_0 = x$ (при n=1: $x^1/1 = x$)
5. Для i от 1 до maxZnach - 1 использует рекуррентную формулу

`Series calculateSumForward(const Series& inputSeries, int maxZnach)` – Прямое суммирование

Принцип работы:

Функция реализует классический алгоритм накопления суммы с вычислением промежуточных погрешностей.

Пошаговый алгоритм:

1. Копирует входную структуру (члены ряда и точное значение)
2. Инициализирует первый элемент суммы и погрешности
3. Для i от 1 до maxZnach - 1:
   - Вычисляет частичную сумму: sums[i] = sums[i-1] + values[i]
   - Вычисляет абсолютную погрешность: errors[i] = |sums[i] - exactValue|
   - Возвращает структуру с заполненными суммами

Направление: От старших членов (с большими значениями) к младшим

`Series calculateSumBackward(const Series& inputSeries, int maxZnach)` – Обратное суммирование

Принцип работы:

Вычисляет частичные суммы ряда, начиная с последнего (самого малого) члена и двигаясь к первому (самому большому), прибавляя предыдущие члены.

Пошаговый алгоритм:

1. Копирует входную структуру
   2. Копируем входные данные в res
   3. Последний элемент:
      `sums[последний] = values[последний]`
   4. Цикл от конца к началу:
      Для i от `maxZnach-2 до 0:`
      `sums[i] = sums[i+1] + values[i]`
      `errors[i] = |sums[i] - exactValue|`
   5. Массивы переворачиваются для правильного порядка вывода.

6. Возвращаем res

```
Series calculateSumBackward(const Series& inputSeries, int maxZnach) {
    Series res = inputSeries;

    // Начинаем с последнего члена ряда
    res.sums[maxZnach - 1] = res.values[maxZnach - 1];
    res.errors[maxZnach - 1] = abs(res.sums[maxZnach - 1] - res.exactValue);

    // Идем с конца к началу, прибавляя предыдущие члены
    for (int i = maxZnach - 2; i >= 0; i--) {
        res.sums[i] = res.sums[i + 1] + res.values[i];
        res.errors[i] = abs(res.sums[i] - res.exactValue);
    }
    std::reverse(res.sums.begin(), res.sums.end());
    std::reverse(res.errors.begin(), res.errors.end());
    return res;
}
```

Рисунок 3 Реализация обратного суммирования

`void printResults(const Series& series, int maxTerms)` - Вывод результатов

Принцип работы:

Функция форматирует и выводит результаты вычислений в удобочитаемом виде.

Пошаговый алгоритм:

1. Устанавливает точность вывода в 10 знаков после запятой
2. Выводит точное значение функции
3. Выводит последнее (наиболее точное) значение суммы
4. Выводит конечную погрешность
5. Формирует таблицу с тремя столбцами:
    - Номер шага (количество слагаемых)
    - Накопленная сумма на каждом шаге
    - Абсолютная погрешность на каждом шаге
6. Использует std::setw() для выравнивания столбцов

Особенности форматирования:

std::fixed - вывод чисел с фиксированной точкой

std::setprecision(10) - 10 знаков после запятой

std::setw(16) - ширина каждого столбца 16 символов

`int main()`- Функция main() является точкой входа в программу и организует весь вычислительный процесс: взаимодействие с пользователем, выбор функции и метода суммирования, вызов соответствующих вычислительных функций и вывод результатов.

Алгоритм работы

1. **Инициализация** – установка русской локали для вывода кириллицы.
2. **Выбор функции** – пользователь вводит число от 1 до 4 для выбора: e^x, sin(x), cos(x) или ln(1+x).
3. **Ввод аргумента** – пользователь задает значение x.
4. **Проверка** – для ln(1+x) проверяется условие x > -1, иначе программа завершается с ошибкой.

5. **Ввод количества членов ряда** – пользователь задает maxTerms.
6. **Выбор метода суммирования** – пользователь выбирает прямой (с начала) или обратный (с конца) порядок суммирования.
7. **Вычисление членов ряда** – через switch вызывается соответствующая функция (calculateEx, calculateSin, calculateCos или calculateLn), которая возвращает структуру Series.
8. **Суммирование** – в зависимости от выбора вызывается calculateSumForward или calculateSumBackward.
9. **Вывод результатов** – функция printResults отображает таблицу со значениями и ошибками.
10. **Завершение** – возврат 0 при успешном выполнении.

```
    std::cin >> sumChoice;
    switch (funcChoice) {
    case 1:
        std::cout << "\nВыбрана функция: e^x";
        result = calculateEx(x, maxTerms);
        break;
    case 2:
        std::cout << "\nВыбрана функция: sin(x)";
        result = calculateSin(x, maxTerms);
        break;
    case 3:
        std::cout << "\nВыбрана функция: cos(x)";
        result = calculateCos(x, maxTerms);
        break;
    case 4:
        std::cout << "\nВыбрана функция: ln(1+x)";
        result = calculateLn(x, maxTerms);
        break;
    default:
        std::cout << "Неверный выбор функции!\n";
        return 1;
    }

    Series finalResult;
    if (sumChoice == 2) {
        finalResult = calculateSumBackward(result, maxTerms);
        std::cout << "\nИспользуется суммирование с конца";
    }
    else {
        finalResult = calculateSumForward(result, maxTerms);
        std::cout << "\nИспользуется суммирование с начала";
    }

    printResults(finalResult, maxTerms);

    return 0;
}
```

Результаты экспериментов

После того как я добился успешной работы моей программы, мною было проведено несколько экспериментов и построил графики. Результаты экспериментов следующие:

```
Используется суммирование с начала
 РЕЗУЛЬТАТЫ
Точное значение функции: 148.4131591026

Последнее значение: 148.4131591026
Погрешность функции: 0.0000000000

Таблица:
Кол-во слагаемых | Значение          | Ошибка
-----------------|-------------------|-------------------
               1 |      1.0000000000 |    147.4131591026
               2 |      6.0000000000 |    142.4131591026
               3 |     18.5000000000 |    129.9131591026
               4 |     39.3333333333 |    109.0798257692
               5 |     65.3750000000 |     83.0381591026
               6 |     91.4166666667 |     56.9964924359
               7 |    113.1180555556 |     35.2951035470
               8 |    128.6190476190 |     19.7941114835
               9 |    138.3071676587 |     10.1059914438
              10 |    143.6894565697 |      4.7237025329
              11 |    146.3806010251 |      2.0325580774
              12 |    147.6038485049 |      0.8093105977
              13 |    148.1135349548 |      0.2996241478
              14 |    148.3095682048 |      0.1035908978
              15 |    148.3795800797 |      0.0335790228
              16 |    148.4029173714 |      0.0102417312
              17 |    148.4102102750 |      0.0029488275
              18 |    148.4123552467 |      0.0008038559
              19 |    148.4129510722 |      0.0002080304
              20 |    148.4131078683 |      0.0000512342
              21 |    148.4131470674 |      0.0000120352
              22 |    148.4131564005 |      0.0000027021
              23 |    148.4131585216 |      0.0000005809
              24 |    148.4131589828 |      0.0000001198
              25 |    148.4131590788 |      0.0000000237
              26 |    148.4131590981 |      0.0000000045
              27 |    148.4131591017 |      0.0000000008
              28 |    148.4131591024 |      0.0000000001
              29 |    148.4131591026 |      0.0000000000
              30 |    148.4131591026 |      0.0000000000
              31 |    148.4131591026 |      0.0000000000
              32 |    148.4131591026 |      0.0000000000
              33 |    148.4131591026 |      0.0000000000
              34 |    148.4131591026 |      0.0000000000
              35 |    148.4131591026 |      0.0000000000
              36 |    148.4131591026 |      0.0000000000
              37 |    148.4131591026 |      0.0000000000
              38 |    148.4131591026 |      0.0000000000
              39 |    148.4131591026 |      0.0000000000
              40 |    148.4131591026 |      0.0000000000
```

```
Используется суммирование с конца
 РЕЗУЛЬТАТЫ
Точное значение функции: 148.4131591026

Последнее значение: 148.4131591026
Погрешность функции: 0.0000000000

Таблица:
Кол-во слагаемых | Значение          | Ошибка
-----------------|-------------------|-------------------
               1 |      0.0000000000 |    148.4131591026
               2 |      0.0000000000 |    148.4131591026
               3 |      0.0000000000 |    148.4131591026
               4 |      0.0000000000 |    148.4131591026
               5 |      0.0000000000 |    148.4131591026
               6 |      0.0000000000 |    148.4131591026
               7 |      0.0000000000 |    148.4131591026
               8 |      0.0000000000 |    148.4131591026
               9 |      0.0000000000 |    148.4131591026
              10 |      0.0000000000 |    148.4131591026
              11 |      0.0000000000 |    148.4131591026
              12 |      0.0000000001 |    148.4131591024
              13 |      0.0000000008 |    148.4131591017
              14 |      0.0000000045 |    148.4131590981
              15 |      0.0000000237 |    148.4131590788
              16 |      0.0000001198 |    148.4131589828
              17 |      0.0000005809 |    148.4131585216
              18 |      0.0000027021 |    148.4131564005
              19 |      0.0000120352 |    148.4131470674
              20 |      0.0000512342 |    148.4131078683
              21 |      0.0002080304 |    148.4129510722
              22 |      0.0008038559 |    148.4123552467
              23 |      0.0029488275 |    148.4102102750
              24 |      0.0102417312 |    148.4029173714
              25 |      0.0335790228 |    148.3795800797
              26 |      0.1035908978 |    148.3095682048
              27 |      0.2996241478 |    148.1135349548
              28 |      0.8093105977 |    147.6038485049
              29 |      2.0325580774 |    146.3806010251
              30 |      4.7237025329 |    143.6894565697
              31 |     10.1059914438 |    138.3071676587
              32 |     19.7941114835 |    128.6190476190
              33 |     35.2951035470 |    113.1180555556
              34 |     56.9964924359 |     91.4166666667
              35 |     83.0381591026 |     65.3750000000
              36 |    109.0798257692 |     39.3333333333
              37 |    129.9131591026 |     18.5000000000
              38 |    142.4131591026 |      6.0000000000
              39 |    147.4131591026 |      1.0000000000
              40 |    148.4131591026 |      0.0000000000
```

Рисунок 4 Результаты для exp(5)

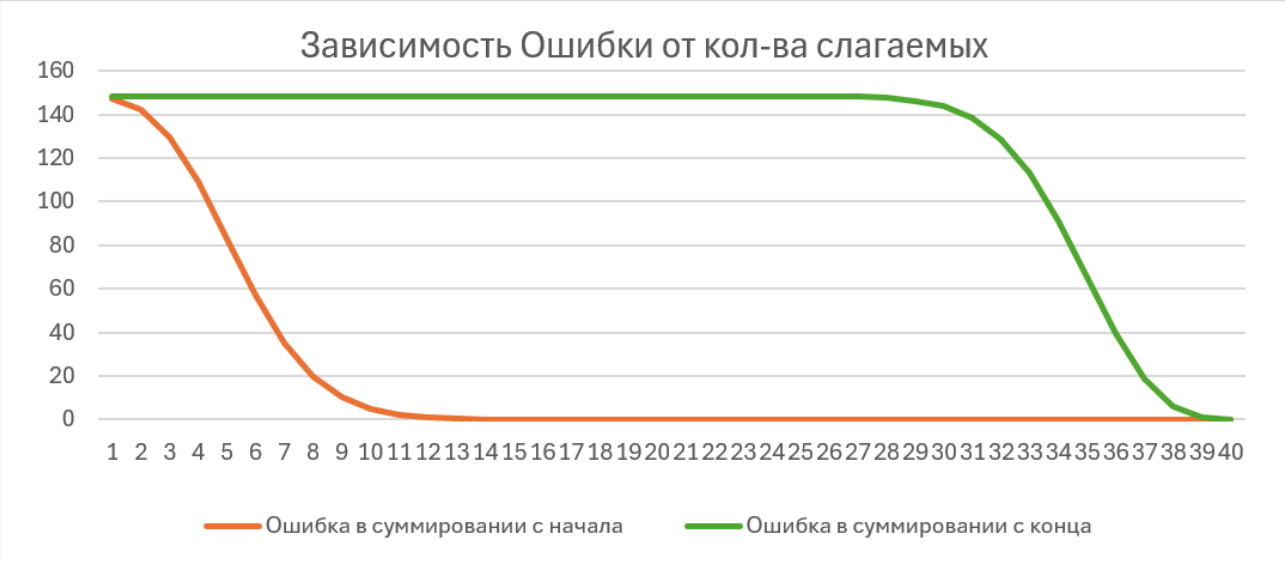

Рисунок 5 График зав-ти отклонения от эталонного значения от количества суммируемых членов ряда экспоненты

```
Используется суммирование с начала
 РЕЗУЛЬТАТЫ
Точное значение функции: -0.9589242747

Последнее значение: -0.9589242747
Погрешность функции: 0.0000000000

Таблица:
```

Кол-во слагаемых	Значение	Ошибка
1	5.0000000000	5.9589242747
2	-15.8333333333	14.8744090587
3	10.2083333333	11.1672576080
4	-5.2926587302	4.3337344555
5	0.0896301808	1.0485544554
6	-1.1336172990	0.1746930243
7	-0.9375840490	0.0213402256
8	-0.9609213407	0.0019970660
9	-0.9587763690	0.0001479056
10	-0.9589331652	0.0000088905
11	-0.9589238321	0.0000004426
12	-0.9589242932	0.0000000185
13	-0.9589242740	0.0000000007
14	-0.9589242747	0.0000000000
15	-0.9589242747	0.0000000000
16	-0.9589242747	0.0000000000
17	-0.9589242747	0.0000000000
18	-0.9589242747	0.0000000000
19	-0.9589242747	0.0000000000
20	-0.9589242747	0.0000000000
21	-0.9589242747	0.0000000000
22	-0.9589242747	0.0000000000
23	-0.9589242747	0.0000000000
24	-0.9589242747	0.0000000000
25	-0.9589242747	0.0000000000
26	-0.9589242747	0.0000000000
27	-0.9589242747	0.0000000000
28	-0.9589242747	0.0000000000
29	-0.9589242747	0.0000000000
30	-0.9589242747	0.0000000000
31	-0.9589242747	0.0000000000
32	-0.9589242747	0.0000000000
33	-0.9589242747	0.0000000000
34	-0.9589242747	0.0000000000
35	-0.9589242747	0.0000000000
36	-0.9589242747	0.0000000000
37	-0.9589242747	0.0000000000
38	-0.9589242747	0.0000000000
39	-0.9589242747	0.0000000000
40	-0.9589242747	0.0000000000

```
Используется суммирование с конца
 РЕЗУЛЬТАТЫ
Точное значение функции: -0.9589242747

Последнее значение: -0.9589242747
Погрешность функции: 0.0000000000

Таблица:
```

Кол-во слагаемых	Значение	Ошибка
1	-0.0000000000	0.9589242747
2	0.0000000000	0.9589242747
3	-0.0000000000	0.9589242747
4	0.0000000000	0.9589242747
5	-0.0000000000	0.9589242747
6	0.0000000000	0.9589242747
7	-0.0000000000	0.9589242747
8	0.0000000000	0.9589242747
9	-0.0000000000	0.9589242747
10	0.0000000000	0.9589242747
11	-0.0000000000	0.9589242747
12	0.0000000000	0.9589242747
13	-0.0000000000	0.9589242747
14	0.0000000000	0.9589242747
15	-0.0000000000	0.9589242747
16	0.0000000000	0.9589242747
17	-0.0000000000	0.9589242747
18	0.0000000000	0.9589242747
19	-0.0000000000	0.9589242747
20	0.0000000000	0.9589242747
21	-0.0000000000	0.9589242747
22	0.0000000000	0.9589242747
23	-0.0000000000	0.9589242747
24	0.0000000000	0.9589242747
25	-0.0000000000	0.9589242747
26	0.0000000000	0.9589242747
27	-0.0000000007	0.9589242740
28	0.0000000185	0.9589242932
29	-0.0000004426	0.9589238321
30	0.0000088905	0.9589331652
31	-0.0001479056	0.9587763690
32	0.0019970660	0.9609213407
33	-0.0213402256	0.9375840490
34	0.1746930243	1.1336172990
35	-1.0485544554	0.0896301808
36	4.3337344555	5.2926587302
37	-11.1672576080	10.2083333333
38	14.8744090587	15.8333333333
39	-5.9589242747	5.0000000000
40	-0.9589242747	0.0000000000

Рисунок 6 Результаты для sin(5)

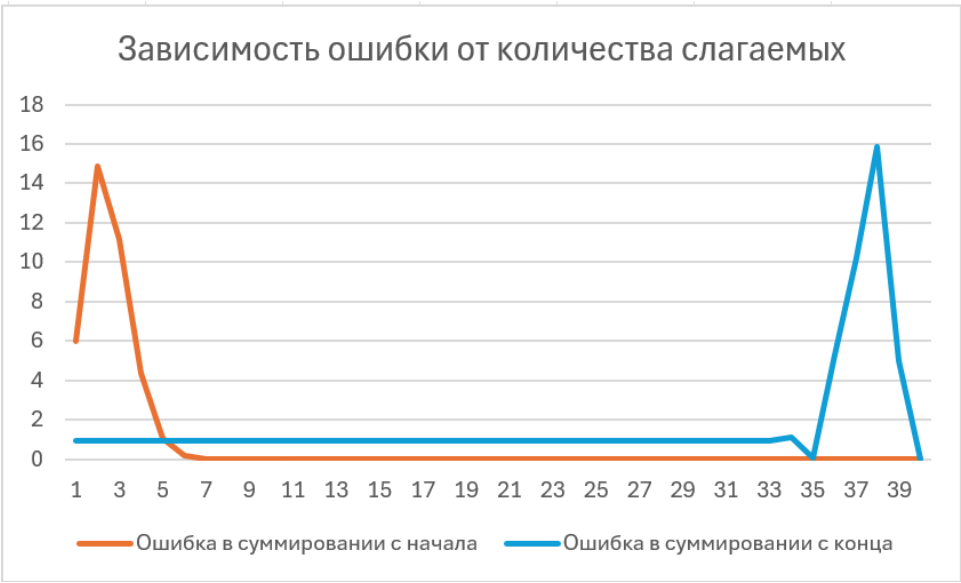

Рисунок 7 График зависимости отклонения от эталонного значения от количества суммируемых членов ряда синуса

```
Выбрана функция: cos(x)
Используется суммирование с начала
 РЕЗУЛЬТАТЫ
Точное значение функции: 0.2836621855

Последнее значение: 0.2836621855
Погрешность функции: 0.0000000000

Таблица:
Кол-во слагаемых | Значение          | Ошибка
-----------------|-------------------|-----------------
               1 |      1.0000000000 |      0.7163378145
               2 |    -11.5000000000 |     11.7836621855
               3 |     14.5416666667 |     14.2580044812
               4 |     -7.1597222222 |      7.4433844077
               5 |      2.5283978175 |      2.2447356320
               6 |     -0.1627466380 |      0.4464088235
               7 |      0.3469398119 |      0.0632776264
               8 |      0.2769279369 |      0.0067342486
               9 |      0.2842208406 |      0.0005586551
              10 |      0.2836250151 |      0.0000371704
              11 |      0.2836642141 |      0.0000020287
              12 |      0.2836620930 |      0.0000000925
              13 |      0.2836621890 |      0.0000000036
              14 |      0.2836621853 |      0.0000000001
              15 |      0.2836621855 |      0.0000000000
              16 |      0.2836621855 |      0.0000000000
              17 |      0.2836621855 |      0.0000000000
              18 |      0.2836621855 |      0.0000000000
              19 |      0.2836621855 |      0.0000000000
              20 |      0.2836621855 |      0.0000000000
              21 |      0.2836621855 |      0.0000000000
              22 |      0.2836621855 |      0.0000000000
              23 |      0.2836621855 |      0.0000000000
              24 |      0.2836621855 |      0.0000000000
              25 |      0.2836621855 |      0.0000000000
              26 |      0.2836621855 |      0.0000000000
              27 |      0.2836621855 |      0.0000000000
              28 |      0.2836621855 |      0.0000000000
              29 |      0.2836621855 |      0.0000000000
              30 |      0.2836621855 |      0.0000000000
              31 |      0.2836621855 |      0.0000000000
              32 |      0.2836621855 |      0.0000000000
              33 |      0.2836621855 |      0.0000000000
              34 |      0.2836621855 |      0.0000000000
              35 |      0.2836621855 |      0.0000000000
              36 |      0.2836621855 |      0.0000000000
              37 |      0.2836621855 |      0.0000000000
              38 |      0.2836621855 |      0.0000000000
              39 |      0.2836621855 |      0.0000000000
              40 |      0.2836621855 |      0.0000000000
```

```
Выбрана функция: cos(x)
Используется суммирование с конца
 РЕЗУЛЬТАТЫ
Точное значение функции: 0.2836621855

Последнее значение: 0.2836621855
Погрешность функции: 0.0000000000

Таблица:
Кол-во слагаемых | Значение          | Ошибка
-----------------|-------------------|-----------------
               1 |     -0.0000000000 |      0.2836621855
               2 |      0.0000000000 |      0.2836621855
               3 |     -0.0000000000 |      0.2836621855
               4 |      0.0000000000 |      0.2836621855
               5 |     -0.0000000000 |      0.2836621855
               6 |      0.0000000000 |      0.2836621855
               7 |     -0.0000000000 |      0.2836621855
               8 |      0.0000000000 |      0.2836621855
               9 |     -0.0000000000 |      0.2836621855
              10 |      0.0000000000 |      0.2836621855
              11 |     -0.0000000000 |      0.2836621855
              12 |      0.0000000000 |      0.2836621855
              13 |     -0.0000000000 |      0.2836621855
              14 |      0.0000000000 |      0.2836621855
              15 |     -0.0000000000 |      0.2836621855
              16 |      0.0000000000 |      0.2836621855
              17 |     -0.0000000000 |      0.2836621855
              18 |      0.0000000000 |      0.2836621855
              19 |     -0.0000000000 |      0.2836621855
              20 |      0.0000000000 |      0.2836621855
              21 |     -0.0000000000 |      0.2836621855
              22 |      0.0000000000 |      0.2836621855
              23 |     -0.0000000000 |      0.2836621855
              24 |      0.0000000000 |      0.2836621855
              25 |     -0.0000000000 |      0.2836621855
              26 |      0.0000000001 |      0.2836621853
              27 |     -0.0000000036 |      0.2836621890
              28 |      0.0000000925 |      0.2836620930
              29 |     -0.0000020287 |      0.2836642141
              30 |      0.0000371704 |      0.2836250151
              31 |     -0.0005586551 |      0.2842208406
              32 |      0.0067342486 |      0.2769279369
              33 |     -0.0632776264 |      0.3469398119
              34 |      0.4464088235 |      0.1627466380
              35 |     -2.2447356320 |      2.5283978175
              36 |      7.4433844077 |      7.1597222222
              37 |    -14.2580044812 |     14.5416666667
              38 |     11.7836621855 |     11.5000000000
              39 |     -0.7163378145 |      1.0000000000
              40 |      0.2836621855 |      0.0000000000
```

Рисунок 8 Результаты для cos(5)

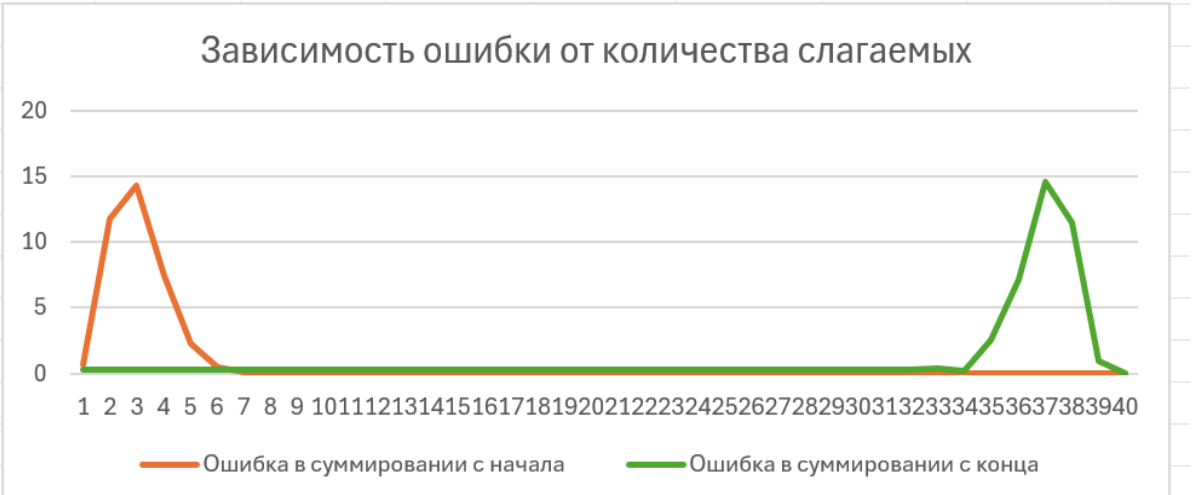

Рисунок 9 График зависимости отклонения от эталонного значения от количества суммируемых членов ряда косинуса

```
Выбрана функция: ln(1+x)
Используется суммирование с начала
 РЕЗУЛЬТАТЫ
Точное значение функции: 0.4054651081

Последнее значение: 0.4054651081
Погрешность функции: 0.0000000000

Таблица:
```

Кол-во слагаемых	Значение	Ошибка
1	0.5000000000	0.0945348919
2	0.3750000000	0.0304651081
3	0.4166666667	0.0112015586
4	0.4010416667	0.0044234414
5	0.4072916667	0.0018265586
6	0.4046875000	0.0007776081
7	0.4058035714	0.0003384633
8	0.4053152902	0.0001498179
9	0.4055323041	0.0000671960
10	0.4054346478	0.0000304603
11	0.4054790370	0.0000139289
12	0.4054586920	0.0000064161
13	0.4054680820	0.0000029739
14	0.4054637223	0.0000013858
15	0.4054657568	0.0000006487
16	0.4054648032	0.0000003049
17	0.4054652520	0.0000001439
18	0.4054650400	0.0000000681
19	0.4054651404	0.0000000323
20	0.4054650927	0.0000000154
21	0.4054651154	0.0000000073
22	0.4054651046	0.0000000035
23	0.4054651098	0.0000000017
24	0.4054651073	0.0000000008
25	0.4054651085	0.0000000004
26	0.4054651079	0.0000000002
27	0.4054651082	0.0000000001
28	0.4054651081	0.0000000000
29	0.4054651081	0.0000000000
30	0.4054651081	0.0000000000
31	0.4054651081	0.0000000000
32	0.4054651081	0.0000000000
33	0.4054651081	0.0000000000
34	0.4054651081	0.0000000000
35	0.4054651081	0.0000000000
36	0.4054651081	0.0000000000
37	0.4054651081	0.0000000000
38	0.4054651081	0.0000000000
39	0.4054651081	0.0000000000
40	0.4054651081	0.0000000000

```
Выбрана функция: ln(1+x)
Используется суммирование с конца
 РЕЗУЛЬТАТЫ
Точное значение функции: 0.4054651081

Последнее значение: 0.4054651081
Погрешность функции: 0.0000000000

Таблица:
```

Кол-во слагаемых	Значение	Ошибка
1	-0.0000000000	0.4054651081
2	0.0000000000	0.4054651081
3	-0.0000000000	0.4054651081
4	0.0000000000	0.4054651081
5	-0.0000000000	0.4054651081
6	0.0000000000	0.4054651081
7	-0.0000000000	0.4054651081
8	0.0000000000	0.4054651081
9	-0.0000000000	0.4054651081
10	0.0000000000	0.4054651081
11	-0.0000000000	0.4054651081
12	0.0000000000	0.4054651081
13	-0.0000000001	0.4054651082
14	0.0000000002	0.4054651079
15	-0.0000000004	0.4054651085
16	0.0000000008	0.4054651073
17	-0.0000000017	0.4054651098
18	0.0000000035	0.4054651046
19	-0.0000000073	0.4054651154
20	0.0000000154	0.4054650927
21	-0.0000000323	0.4054651404
22	0.0000000681	0.4054650400
23	-0.0000001439	0.4054652520
24	0.0000003049	0.4054648032
25	-0.0000006487	0.4054657568
26	0.0000013858	0.4054637223
27	-0.0000029739	0.4054680820
28	0.0000064161	0.4054586920
29	-0.0000139289	0.4054790370
30	0.0000304603	0.4054346478
31	-0.0000671960	0.4055323041
32	0.0001498179	0.4053152902
33	-0.0003384633	0.4058035714
34	0.0007776081	0.4046875000
35	-0.0018265586	0.4072916667
36	0.0044234414	0.4010416667
37	-0.0112015586	0.4166666667
38	0.0304651081	0.3750000000
39	-0.0945348919	0.5000000000
40	0.4054651081	0.0000000000

Рисунок 10 Результаты для ln(1.5)

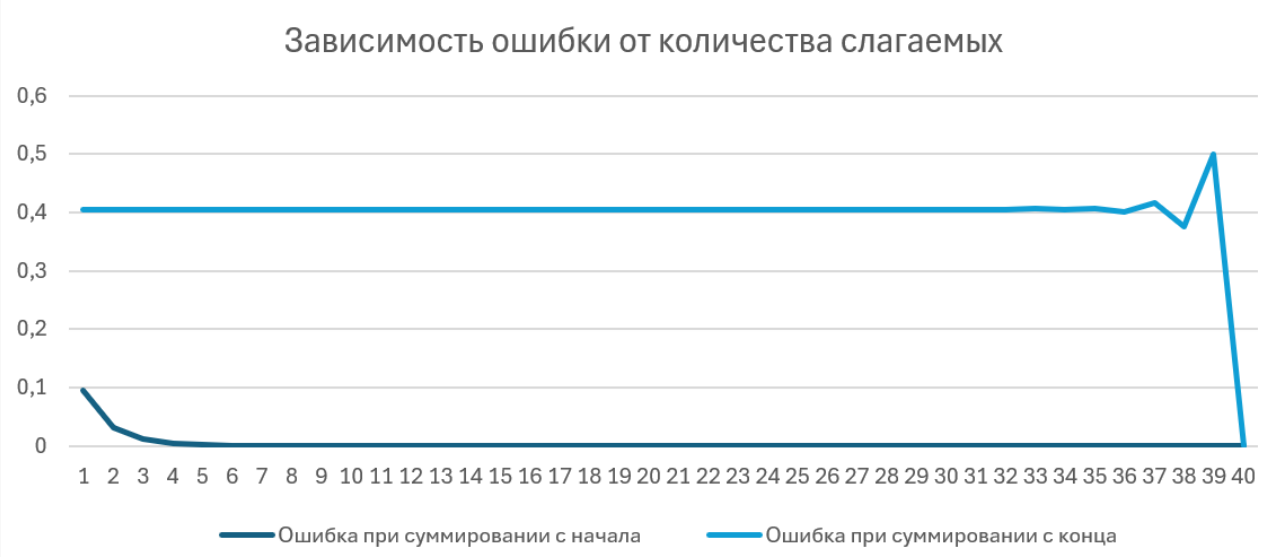

Рисунок 11 График зависимости отклонения от эталонного значения от количества суммируемых членов ряда логарифма

Проанализировав результаты экспериментов мы можем сделать вывод, что программа работает корректно. Оба метода сходятся к одному результату – при достаточном количестве членов ошибка стремится к нулю. Для e^x и ln(1+x) – разница между методами минимальна, можно использовать любой метод суммирования Для sin(x) и cos(x) – обратный метод дает меньшую ошибку, особенно при большом количестве слагаемых. Это происходит потому что синус и косинус — это ряды с чередованием плюсов и минусов. Прямой метод заставляет вас постоянно вычитать близкие числа, что убивает точность. Обратный метод складывает сначала мелкие члены, поэтому точность сохраняется. Прямой метод проще в реализации но при знакопеременных рядах может накапливать погрешность из-за вычитания близких чисел. Выходит что выбор способа суммирования влияет на точность: для знакопеременных рядов предпочтительнее обратный порядок, для знакопостоянных подходит любой.

Заключение

Я написал рабочую программу на языке “C++” которая успешно вычисляет функции e^x, sin(x), cos(x) и ln(1+x) с помощью разложения в ряд Тейлора. Были реализованы два способа суммирования членов ряда: прямой и обратный. Так же я реализовал удобный интерфейс и вывод результатов в виде таблицы. По результатам экспериментов я сделал выводы об эффективности способов суммирования.

Литература

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.
2. Кнут Д. Э. Искусство программирования: Сортировка и поиск. – Издательский дом Вильямс, 2000. – Т. 3.
3. Скиена С. С. Алгоритмы. Руководство по разработке. — СПб.: БХВ-Петербург, 2014.

Приложение

```
void printResults(const Series& series, int maxTerms) {
    std::cout << std::fixed << std::setprecision(10);
    std::cout << "\n РЕЗУЛЬТАТЫ \n";
    std::cout << "Точное значение функции: " << series.exactValue << "\n\n";
    std::cout << "Последнее значение: " << series.sums[maxTerms - 1] << "\n";
    std::cout << "Погрешность функции: " << series.errors[maxTerms - 1] << "\n\n";

    std::cout << "Таблица:\n";
    std::cout << "Кол-во слагаемых | Значение         | Ошибка\n";
    std::cout << "-----------------|------------------|------------------\n";

    for (int i = 0; i < maxTerms; i++) {
        std::cout << std::setw(16) << (i + 1) << " | ";
        std::cout << std::setw(16) << series.sums[i] << " | ";
        std::cout << std::setw(16) << series.errors[i] << "\n";
    }
}
```

Рисунок 12 Функция вывода и форматирования результатов

```
int main() {
    setlocale(LC_ALL, "Russian");

    double x;
    int maxTerms;
    int funcChoice;
    int sumChoice;
    Series result;

    // Выбор функции
    std::cout << " ВЫЧИСЛЕНИЕ ФУНКЦИЙ ЧЕРЕЗ РЯД ТЕЙЛОРА \n";
    std::cout << "Выберите функцию для вычисления:\n";
    std::cout << "1 - e^x\n";
    std::cout << "2 - sin(x)\n";
    std::cout << "3 - cos(x)\n";
    std::cout << "4 - ln(1+x)\n";
    std::cout << "Ваш выбор: ";
    std::cin >> funcChoice;

    // Ввод x
    std::cout << "\nВведите x: ";
    std::cin >> x;



    std::cout << "Введите максимальное количество членов ряда: ";
    std::cin >> maxTerms;

    // Выбор способа суммирования
    std::cout << "\nВыберите способ суммирования:\n";
    std::cout << "1 - Суммирование с начала (прямой порядок)\n";
    std::cout << "2 - Суммирование с конца (обратный порядок)\n";
    std::cout << "Ваш выбор: ";
    std::cin >> sumChoice;
    switch (funcChoice) {
    case 1:
        std::cout << "\nВыбрана функция: e^x";
        result = calculateEx(x, maxTerms);
        break;
    case 2:
        std::cout << "\nВыбрана функция: sin(x)";
        result = calculateSin(x, maxTerms);
        break;
    case 3:
        std::cout << "\nВыбрана функция: cos(x)";
        result = calculateCos(x, maxTerms);
        break;
    case 4:
        std::cout << "\nВыбрана функция: ln(1+x)";
        result = calculateLn(x, maxTerms);
        break;
    default:
        std::cout << "Неверный выбор функции!\n";
        return 1;
    }

    Series finalResult;
    if (sumChoice == 2) {
        finalResult = calculateSumBackward(result, maxTerms);
        std::cout << "\nИспользуется суммирование с конца";
    }
    else {
        finalResult = calculateSumForward(result, maxTerms);
        std::cout << "\nИспользуется суммирование с начала";
    }

    printResults(finalResult, maxTerms);

    return 0;
}
```

Рисунок 13 Функция int main()